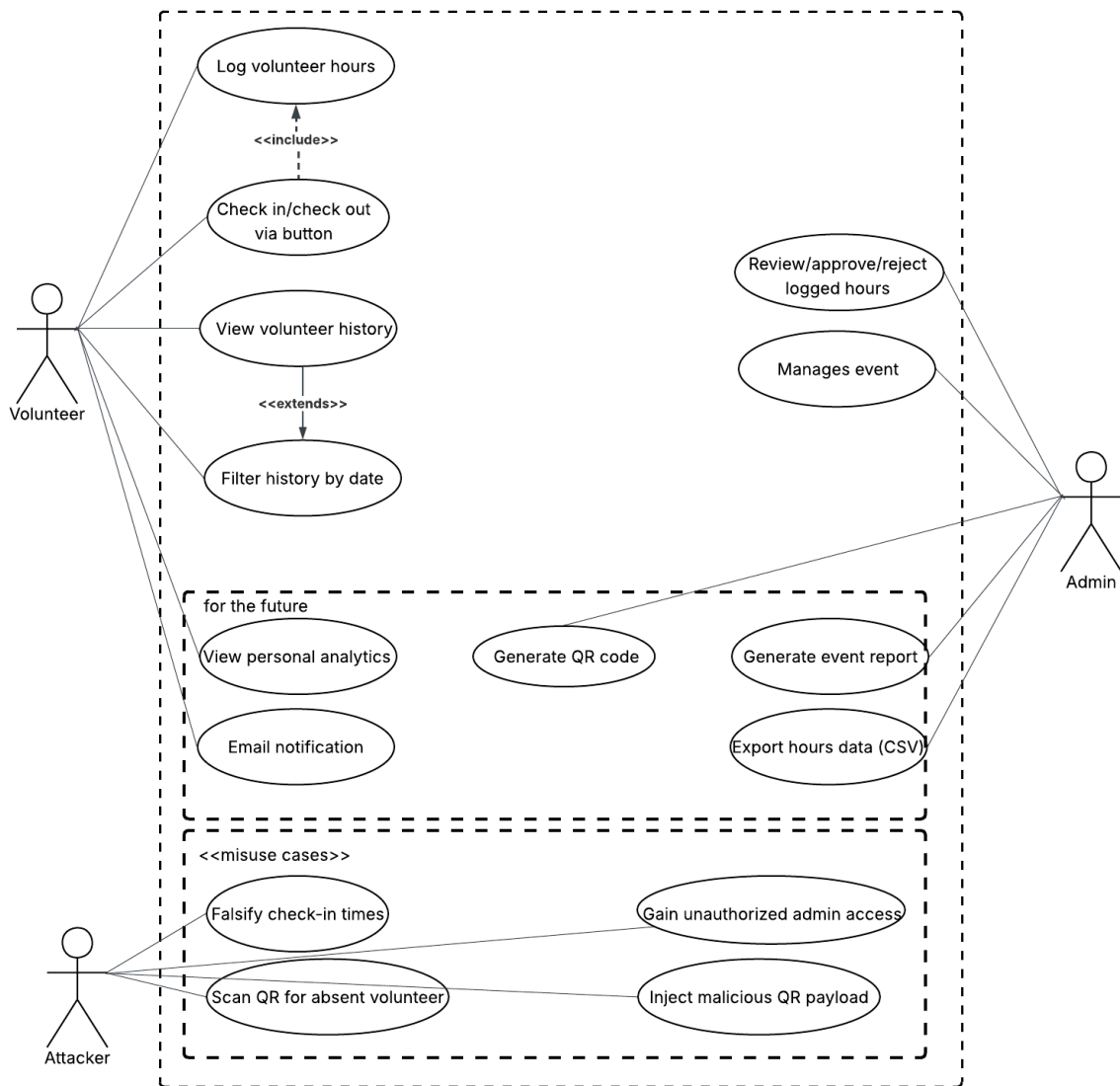


Use Case Diagram



- *Moved QR code check in/check out (partially implemented as button triggered time based check in instead) to for future use*

1. Log Volunteer Hours

Use case:	Log Volunteer Hours
-----------	---------------------

Primary actor	Volunteer
Goal in context:	To record volunteer hours at an event by scanning QR codes at check-in and check out
Preconditions	A volunteer is registered in the system. An event with active QR codes exists and the event is in progress
Trigger:	The volunteer and scans the entry QR code at the event
Scenario:	1. Volunteers open the QR scanner page on their device or the device provided at the event 2. Volunteers scan the entry QR code displayed at the event. 3. The system sends a login request containing volunteerID and eventID to the Hours Controller. 4. Hours Controller queries the database to check for an active event for a volunteer 5. Database checks for existing event, returns null for new events 6. The Hours Controller creates a new VolunteerHoursRecord with the volunteer's ID, event ID, and login time 7. Database returns a recordID confirming the event was created 8. The system displays a check in confirmation message to the volunteer 9. At the end of the event, the volunteer scans the exit QR code 10. The system sends a logout request with the volunteerID and eventID to the Hours Controller 11. Hours Controller queries the database to find the active event 12. Database returns the existing event record 13. Hours Controller calculates hours and updates logout time and hours. 14. The system sets the record status to pending approval 15. The system displays a summary showing hours logged and pending approval status
Exceptions:	1. QR code is expired or belongs to a cancelled event, System displays an error message and check-in is rejected 2. Volunteer attempts to check out without a prior check in, System displays an error message no active check in found, no record is updated 3. QR scanner is unavailable or fails to read the code, Volunteer may retry, event staff can manually log hours 4. Volunteer scans a QR code for an event they are not registered for, System displays an error and does not create a record
Misuse/Abuse Cases:	1. Scan QR for absent volunteer, a participant scans for someone who is not present to falsely record their hours Mitigation, Admin reviews pending hours and cross-references attendance records 2. Inject malicious QR payload, a malformed QR attempts SQL injection or script injection Mitigation, all QR input is validated and sanitized server-side before any database operation.

2. Checkin/Checkout Using QR Code

Use case:	Checkin/Checkout Using QR Code
Primary actor	Volunteer
Goal in context:	To use QR code scanning as the mechanism for recording arrival and departure at a volunteer event
Preconditions	The event is in progress, the event's QR codes are active, the volunteer is authenticated in the system
Trigger:	The volunteer physically arrives at or departs from the event location and scans the posted QR code
Scenario:	1. Volunteer navigates to the QR scanner interface on a device 2. Volunteers position cameras or scanners to read the QR code displayed at the event entrance 3. Scanner decodes the QR and extracts the eventID and hash 4. The system sends the token to the Hours Controller for validation 5. Hours Controller validates the token (checks expiry, event status, and signature) 6. If valid, the system proceeds with check-in (creates a VolunteerHoursRecord) or check-out (updates the record with logout time and hours) 7. The system confirms the action to the volunteer with a visual message (Checked In or Checked Out X hours logged)
Exceptions:	1. QR code is unreadable, volunteer retries, if persistent, event staff manually logs the check-in or check-out 2. Token is invalid or has expired, system rejects the scan and displays Invalid QR Code, volunteer contacts event staff 3. Event is not in progress, System rejects the scan and informs the volunteer the event is closed 4. Volunteer is already checked in and scans entry QR again, system warns that the volunteer is already checked in 5. Device camera or scanner malfunctions volunteer reports to event staff for manual entry
Misuse/Abuse Cases:	1. Scan QR for absent volunteer, a participant scans for someone who is not present to falsely record their hours Mitigation, admin reviews pending hours and cross-references attendance records 2. Inject malicious QR payload a malformed QR attempts SQL injection or script injection Mitigation, all QR input is validated and sanitized server-side before any database operation

3. View Volunteer History

Use case:	View Volunteer History
Primary actor	Volunteer

Goal in context:	To allow a volunteer to view their complete history of approved volunteer hours across all events
Preconditions	The volunteer is logged into the system. At least one approved volunteer hours record exists for the volunteer
Trigger:	The volunteer navigates to the volunteer history or tracker tab in the application
Scenario:	1. Volunteer clicks on the history or tracker tab in the navigation menu 2. Tracker UI sends a fetch history request to the History Controller with the volunteer's ID 3. History Controller queries the database for all approved records associated with the volunteerID 4. Database returns the list of approved records 5. History Controller calculates aggregate totals (past month, past year, all time) 6. History Controller returns history data and totals to the Tracker UI 7. System renders the history list displaying event name, date, hours logged, and status for each entry 8. System displays summary totals
Exceptions:	1. No approved records exist for the volunteer, system displays a message no volunteer history found with a prompt to attend an event 2. Database query fails, system displays a generic error message and logs the failure for debugging 3. Volunteer's event has expired, system redirects to the login page.
Misuse/Abuse Cases:	1. Volunteer attempts to access another volunteer's history by manipulating URL parameters, system enforces event based authorization, only records matching the authenticated volunteer's ID are returned 2. Automated scraping of history data, system implements rate limiting to prevent bulk data extraction

4. Filter History By Date

Use case:	Filter History By Date
Primary actor	Volunteer
Goal in context:	To allow a volunteer to filter their volunteer history to a specific date range (past month, past year, or all time)
Preconditions	The volunteer is on the volunteer history page at least one approved record exists.
Trigger:	The volunteer selects a filter option (past month, past year, or all time) on the history page.
Scenario:	1. Volunteers view their full history list on the Tracker UI 2. Volunteer selects a filter from the available filter options 3. Tracker UI sends a filter history request to the History Controller with the volunteerID and the selected date range 4. History Controller queries the database for approved records within the

	specified date range for that volunteer 5. Database returns the filtered records 6. History Controller recalculates totals for the selected range 7. System updates the display to show only the filtered records and the updated totals
Exceptions:	1. No records exist in the selected date range system displays no records found for this period 2. Invalid date range provided system defaults to all time and displays a warning 3. Database query for the filtered range fails system displays a generic error message
Misuse/Abuse Cases:	1. Volunteer attempts to inject a malicious date range string to manipulate the query system validates and parameterizes all query inputs before sending to the database to prevent SQL injection

5. Review/Approve/Reject Logged Hours

Use case:	Review/Approve/Reject Logged Hours
Primary actor	Admin
Goal in context:	To allow an admin to review volunteer hours submitted for an event and approve or reject each record
Preconditions	An event has been completed and one or more VolunteerHoursRecords with status pending exist. The admin is logged into the system
Trigger:	The admin opens the Manage Hours page for a specific event
Scenario:	1. Admin navigates to the manage hours section and selects an event 2. Admin UI sends a fetch pending request to the Approval Controller with the eventID 3. Approval Controller queries the database for all records with status pending for that event 4. Database returns the pending records 5. Approval Controller verifies attendance data using volunteer IDs 6. System renders a review table listing each volunteer, their check-in/check-out times, and calculated hours 7. Admin reviews the table and clicks approve on a record 8. Admin UI sends an approve hours request to the Approval Controller with the recordID 9. Approval Controller updates the record status to approved in the database 10. Database confirms the update 11. System displays hours approved confirmation on the Admin UI 12. The approved hours become visible in the volunteer's history
Exceptions:	1. Admin rejects/flags hours, admin clicks reject and provides a reason. Approval Controller sets status to rejected with the reason stored. Volunteer is notified and rejected hours are not counted

	2. No pending records exist for the event. system displays no pending records to review 3. Database update fails during approval, system displays an error and retries or prompts admin to try again 4. Admin attempts to approve hours for a cancelled event, system blocks the action and displays an appropriate message
Misuse/Abuse Cases:	1. Gain unauthorized admin access a regular volunteer attempts to access the admin review panel Mitigation, admin routes are protected by role-based authentication, volunteers are redirected if they attempt to access admin pages 2. Admin approves hours fraudulently without verification, system logs all approval actions with a timestamp and admin ID for audit purposes

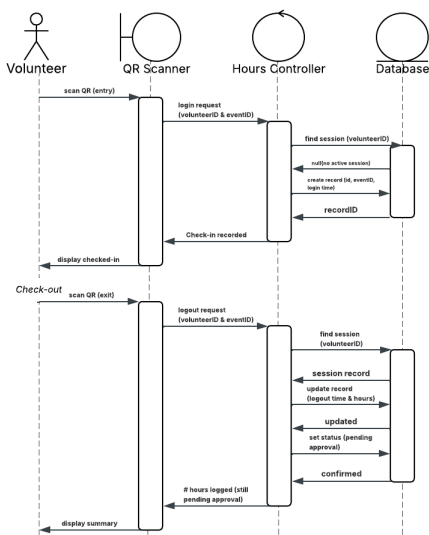
6. Manages Event

Use case:	Manages Event
Primary actor	Admin
Goal in context:	To allow an admin to create, manage, and archive volunteer events, including generating QR codes for check-in and check-out
Preconditions	The admin is authenticated. For editing or deleting, the event must already exist in the system
Trigger:	The admin opens the event management interface and begins creating a new event or editing an existing one
Scenario:	1. Admin navigates to the events section of the admin dashboard 2. Admin clicks create new event and fills out the event form with name, date, organization, and max slots 3. Admin submits the event form 4. History Controller validates and saves the event data to the database 5. Database generates and returns the eventID 6. The system automatically triggers QR code generation for the new event 7. Database stores the QR code URLs and returns them to the controller 8. The system displays the event details and QR codes for the admin to download or print 9. Event status is set to published QR codes are active and visible to volunteers
Exceptions:	1. Duplicate event name, admin submits an event with a name that already exists. System returns an error event name already exists, admin must choose a different name 2. Missing required fields, system highlights missing fields and prevents submission 3. Admin edits an event that is already in progress, system warns admin that changes may affect active check ins 4. Admin deletes a Published event event is marked deleted and QR codes are invalidated 5. Admin cancels an event event is marked cancelled any existing pending records

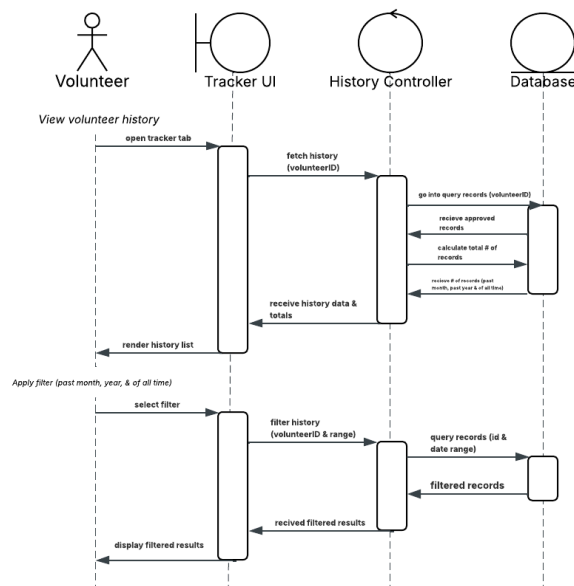
	are flagged for admin review
Misuse/Abuse Cases:	1. Gain unauthorized admin access to event management, a volunteer or attacker attempts to access event creation endpoints directly Mitigation, all event management endpoints require admin authentication 2. Inject malicious QR payload via event creation, an attacker supplies a malicious event name or URL to manipulate QR code content Mitigation, all inputs are sanitized and QR code content is generated server-side from validated data only

Sequence Diagram

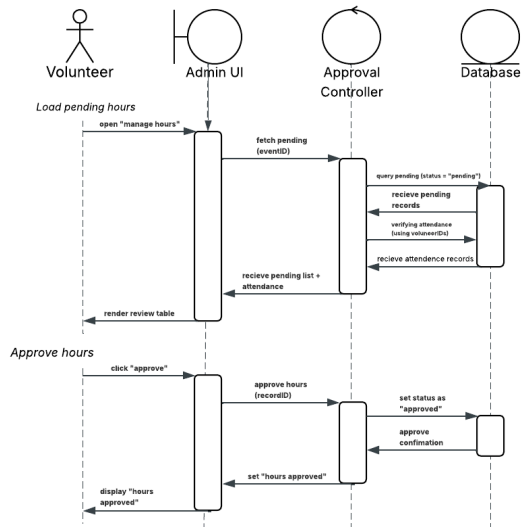
Log Volunteer Hours



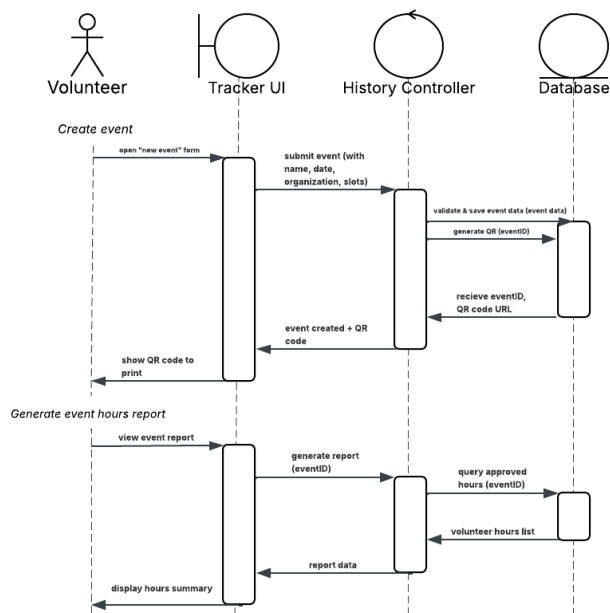
View Volunteer History



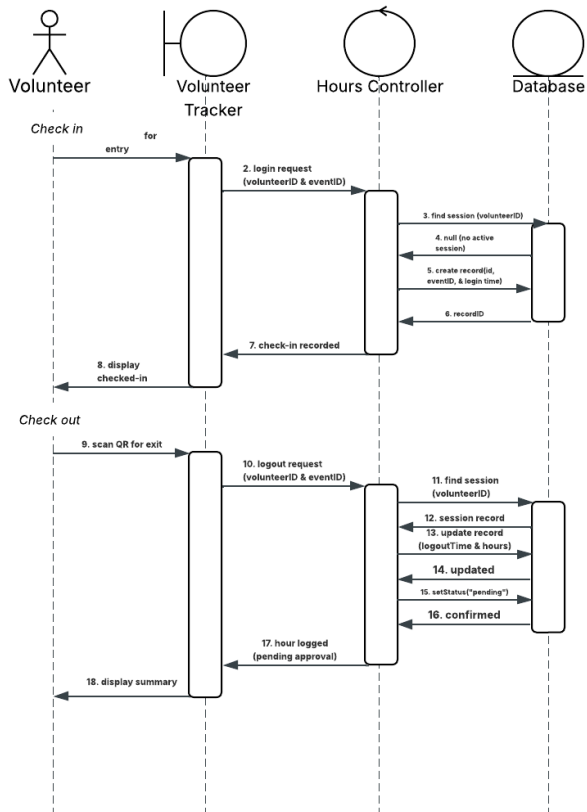
Admin reviews and approves logged hours



Admin manages events

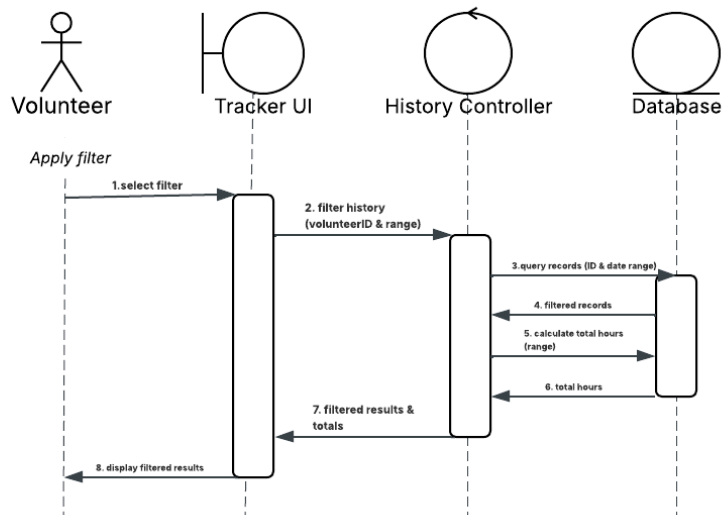


Check in / check out



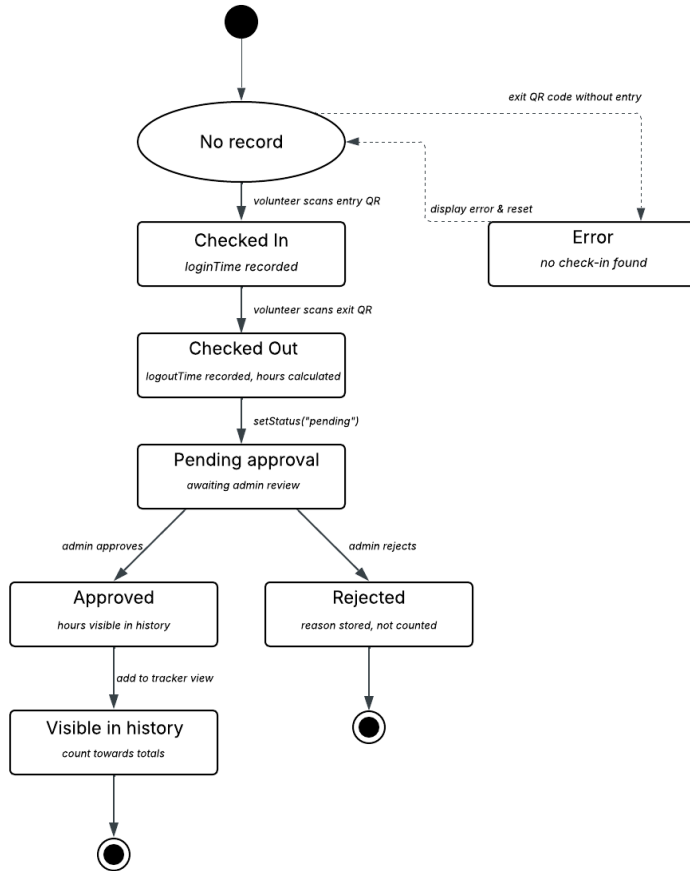
Changed for without QR code

Filter volunteer history by date

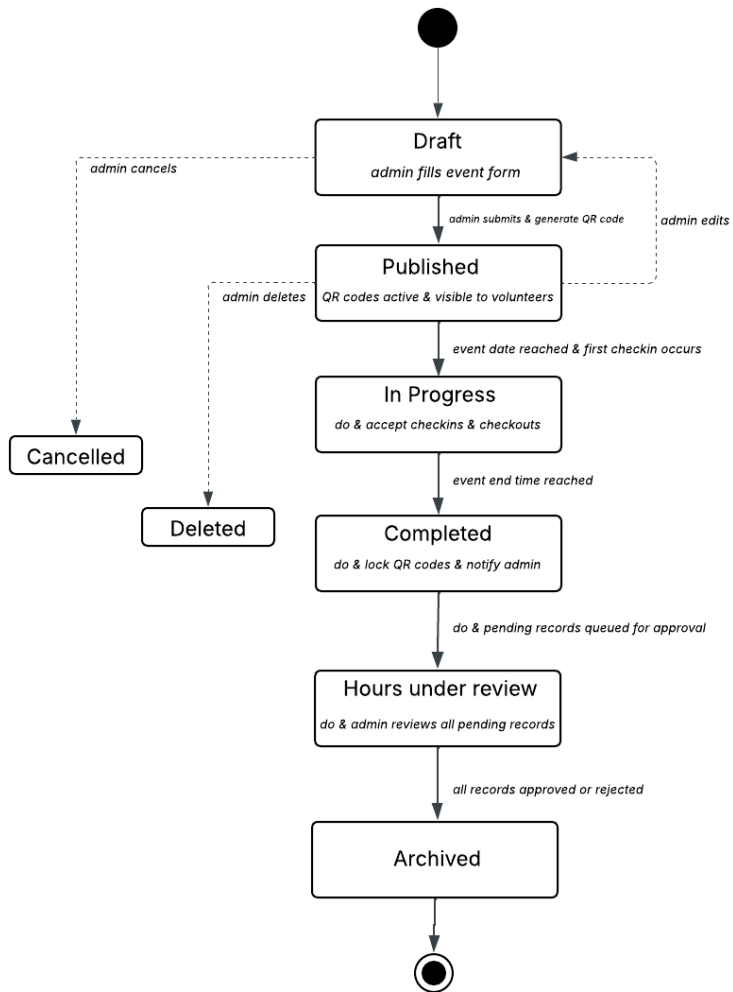


State Diagram

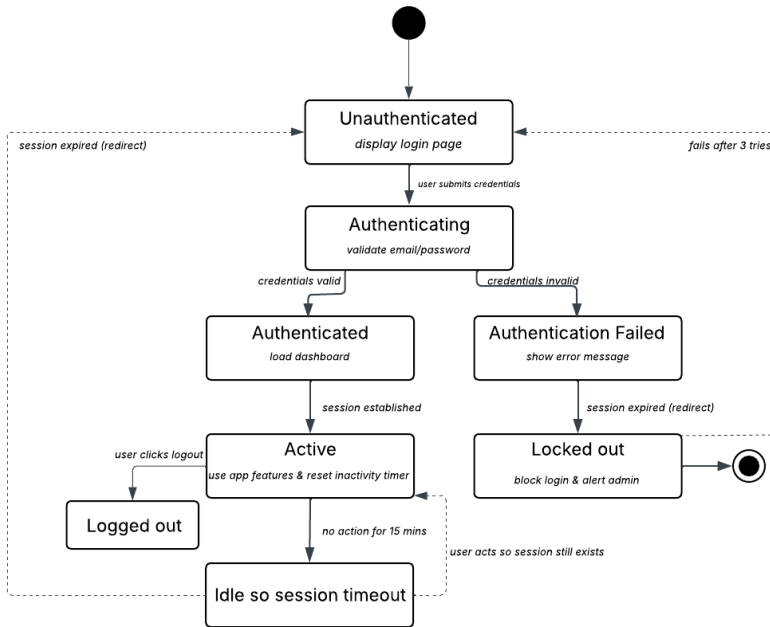
VolunteerHoursRecord



Event

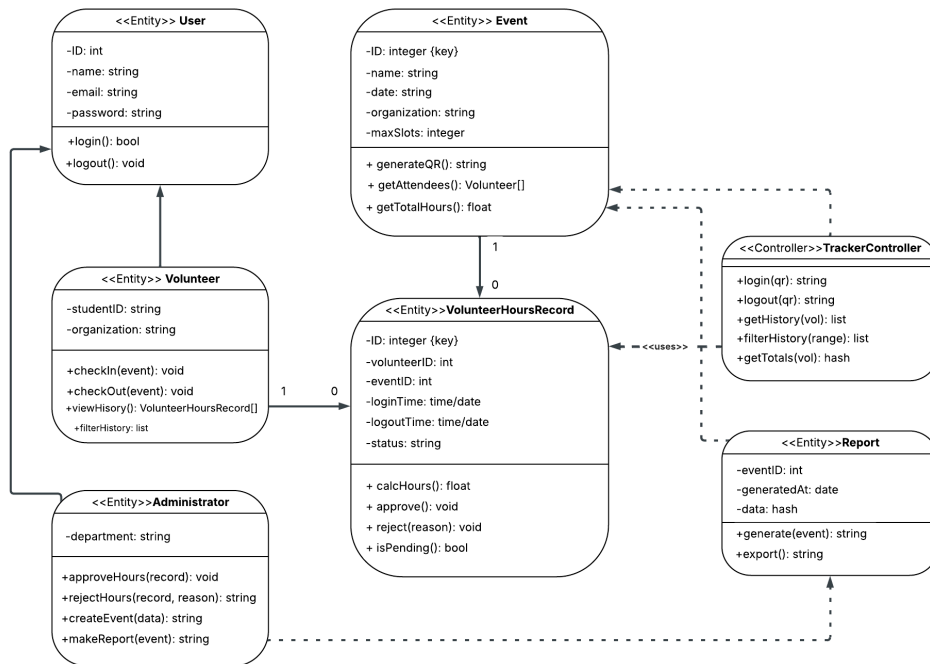


User Session



Detailed Design Class Diagram (got rid of QR code)

Class Diagram





UI design mock-ups and discussion

Volunteer History

Choose a time range:

Total Hours: 0

Event Name	Date (XX/XX/XXXX)	Hours	Status
Name	XX/XX/XXXX	#	Approved/Rejected/ Pending
Name	XX/XX/XXXX	#	Approved/Rejected/ Pending
Name	XX/XX/XXXX	#	Approved/Rejected/ Pending
Name	XX/XX/XXXX	#	Approved/Rejected/ Pending
Name	XX/XX/XXXX	#	Approved/Rejected/ Pending

Event Actions

Status:

Month

Year

All Time

Approved

Rejected

Pending

The Volunteer History UI mockup has a filter for month, year, and of all time, the total hours table with the event name, date, and hours, the check-in and check-out for event actions, and the approve hours buttons with the status that could be approved, rejected, or pending done by admin.

Admin Event Approval

http://SERVD.com

Event Name

User	Event	Date	Hours	Status	Approve	
Name	Event Name	XX/XX/XXXX, XX:XX:XX AM/PM	##	Pending	Approve	Reject
Name	Event Name	XX/XX/XXXX, XX:XX:XX AM/PM	##	Pending	Approve	Reject
Name	Event Name	XX/XX/XXXX, XX:XX:XX AM/PM	##	Pending	Approve	Reject
Name	Event Name	XX/XX/XXXX, XX:XX:XX AM/PM	##	Pending	Approve	Reject
Name	Event Name	XX/XX/XXXX, XX:XX:XX AM/PM	##	Pending	Approve	Reject

The Admin Event Approval UI mockup is for the admin to approve or reject each student's hours based on their check-in and check-out times based on the events.